

Jet Engine Hospital: A Multi-Task Early-Warning System for NASA C-MAPSS Turbofan Engines

Amirali Nasiri Vand

ID: 402222150

Summer 2026

Abstract

This report presents an end-to-end predictive-maintenance system for NASA’s C-MAPSS turbofan engine benchmark. At every engine cycle the system outputs a Remaining Useful Life (RUL) point estimate with an uncertainty band, calibrated failure probabilities at 10-, 20-, and 30-cycle horizons, and an unsupervised anomaly score, fusing all three into an auditable CONTINUE/INSPECT/STOP recommendation. The pipeline is built on FD001 (single condition, single fault mode) as the foundation stage and stress-tested on FD003 (single condition, two simultaneous fault modes) as the Stage-2 challenge, under one identical, leakage-safe, engine-level evaluation protocol. Gradient Boosting is selected for RUL regression, calibrated Logistic Regression for horizon classification, and validation-tuned Local Outlier Factor / Isolation Forest for anomaly detection on FD001 / FD003 respectively. Two engineering challenges are documented: choosing a defensible RUL cap and its consequence for the FD003 decision policy, and deferring the FD004 bonus dataset once it proved by far the most computationally expensive benchmark under the shared protocol.

1 Problem Framing

1.1 Mission

For engine i at cycle t , let $x(i, t)$ denote the three operating settings and 21 sensor readings observed up to and including cycle t . From $x(i, t)$ alone (never future cycles), the system must produce:

- (1) **Regression.** $\text{RUL}(i, t) = T(i) - t$, where $T(i)$ is engine i ’s failure cycle in a run-to-failure trajectory.
- (2) **Classification.** For each horizon $h \in \{10, 20, 30\}$, a calibrated probability that $y(i, t, h) = \mathbb{1}[\text{RUL}(i, t) \leq h]$.
- (3) **Unsupervised detection.** An anomaly score $a(i, t)$ fitted without failure labels, rising consistently as an engine approaches failure.

These three signals are fused at inference time into one of three maintenance actions – CONTINUE, INSPECT, STOP – through a declared, auditable decision policy (Section 7).

1.2 Core constraint

The train/validation/test split is made *by engine ID*, before any preprocessing is fit, feature is selected, window is built, or threshold is tuned. Every window belonging to one engine lives in exactly one split; this shapes essentially every design decision below.

2 Dataset and Data Audit

2.1 Benchmark subsets used

Two of the four official C-MAPSS subsets build the two-stage comparison: FD001 as the mandatory foundation, and FD003 as the Stage-2 challenge (chosen over FD002 because it stresses the *multi-fault-mode* axis rather than the operating-condition axis, keeping one operating condition constant so the two fault modes are the only new source of difficulty over FD001).

Table 1: Official benchmark subsets used in this project.

Subset	Train engines	Test engines	Conditions	Fault modes	Role
FD001	100	100	1 (sea level)	1: HPC	Stage 1 – foundation
FD003	100	100	1 (sea level)	2: HPC + fan	Stage 2 – multi-fault challenge

Each row is one engine-cycle snapshot with 26 whitespace-separated fields (`engine_id`, `cycle`, three operating settings, 21 sensor channels). Training trajectories run to failure; test trajectories are truncated before failure, with true remaining life supplied in a companion `RUL_FD00x.txt` file. Loading asserts that this file’s engine count matches the number of distinct test engines before anything downstream runs.

2.2 Row- and engine-level audit

Table 2 summarizes the audit performed on training engines only (never on test). FD003 engines run noticeably longer, with a much heavier right tail before failure, than FD001 – a fact that matters directly for the RUL-cap discussion in Section 13. Figure 1 illustrates the engine lifetime distribution for FD001 training engines.

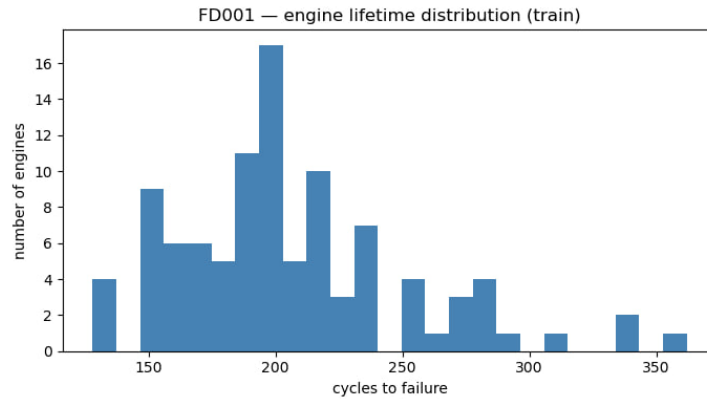


Figure 1: FD001 engine lifetime distribution (training set).

Table 2: Data audit summary (training engines only).

	FD001	FD003
Engines	100	100
Rows	20,631	24,720
Lifetime mean / std (cycles)	206.3 / 46.3	247.2 / 86.5
Lifetime min / max (cycles)	128 / 362	145 / 525
Uncapped RUL, 90th / 99th pct.	198 / 287	273 / 441
Near-constant sensors dropped	6	5
Duplicate (<code>engine_id</code> , <code>cycle</code>) keys	0	0
Non-finite values after parsing	0	0

Six sensors in FD001 (`sensor_1`, 5, 10, 16, 18, 19) and five in FD003 (`sensor_1`, 5, 16, 18,

19) have training standard deviation below 10^{-3} and are dropped before feature construction; **sensor_10** carries a small but non-trivial signal in FD003 and is kept there.

Furthermore, Figure 2 displays the sample sensor trajectories for select FD003 engines, highlighting the increased variance and variance spread resulting from two distinct fault modes.

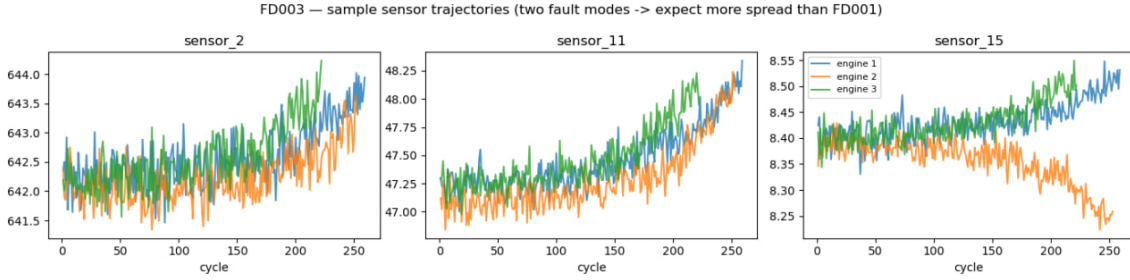


Figure 2: FD003 sample sensor trajectories for representative engines showing increased variance due to multiple fault modes.

2.3 Class balance after the engine-level split

Training engines are split 80/20 into train and validation *by engine ID* (seed 42), and only then are classification labels computed and inspected for balance:

Table 3: Positive rate per horizon after the engine-level split (training engines).

Dataset	$h = 10$	$h = 20$	$h = 30$
FD001	0.053	0.102	0.150
FD003	0.044	0.085	0.125

The shorter horizons are heavily imbalanced (roughly 1 positive row in 20 at $h = 10$), which is why precision–recall analysis and calibrated probabilities – rather than accuracy or an unexamined 0.50 threshold – drive every classification decision downstream.

3 Leakage Controls

Table 4: Leakage controls implemented at each pipeline step.

Step	Allowed data	Control implemented
Split IDs	engine_id only	Train/val/test engine ID sets asserted disjoint (<code>splits.assert_disjoint</code>) and persisted to disk.
Fit preprocessing	Training engines	The <code>StandardScaler</code> inside <code>FeaturePipeline</code> is fit once, only on training rows.
Create windows	Within each split	Rolling statistics computed per-engine (<code>groupby("engine_id")</code>); no window spans two engines or two splits.
Tune / calibrate	Validation engines	RUL cap, classifier thresholds, Platt calibration, and decision-policy quantiles are all read off validation predictions only.
Final evaluation	Test engines	Test engines are touched once, through the locked pipeline, for the master results table.

A dedicated *causality test* makes the “no future information” guarantee executable: it perturbs every sensor value *after* a chosen cycle t by a large additive shift and asserts the feature vector computed *at* t is bit-for-bit unchanged. All rolling statistics are trailing by construction (`pandas` rolling/expanding windows, `min_periods=1`, never centered), so the test passes: `causality_test passed: True`.

4 Feature Engineering

Let $s_c(i, t)$ denote sensor channel c for engine i at cycle t , restricted to the k kept sensors. For each kept sensor, using only cycles $\leq t$:

$$\text{current value: } s_c(i, t) \quad (1)$$

$$\text{trailing rolling mean/std, window } w : \mu_{c,w}(i, t) = \frac{1}{|W_t|} \sum_{\tau \in W_t} s_c(i, \tau), \quad \sigma_{c,w}(i, t) = \text{std}_{\tau \in W_t} s_c(i, \tau) \quad (2)$$

$$\text{trailing slope over the largest window } w^* : \text{slope}_c(i, t) = \frac{s_c(i, t) - s_c(i, t - w^*)}{w^*} \quad (3)$$

$$\text{EWMA (span 10): } \text{ewma}_c(i, t) = \alpha s_c(i, t) + (1 - \alpha) \text{ewma}_c(i, t - 1), \quad \alpha = \frac{2}{10+1} \quad (4)$$

$$\text{first difference: } \Delta s_c(i, t) = s_c(i, t) - s_c(i, t - 1) \quad (5)$$

$$\text{expanding-mean residual: } r_c(i, t) = s_c(i, t) - \frac{1}{t} \sum_{\tau=1}^t s_c(i, \tau) \quad (6)$$

where $W_t = \{\tau : \tau \leq t, \tau > t - w\}$, $w \in \{5, 10, 20\}$ for the mean/std family. Features are assembled per engine (no window spans an engine boundary), then scaled with a `StandardScaler` fit on training rows only.

Table 5: Feature matrix dimensions after engineering.

Dataset	Kept sensors	Engineered features	Train / Val / Test rows
FD001	15	255	16,779 / 3,852 / 13,096
FD003	16	272	20,074 / 4,646 / 16,596

Ridge uses current-cycle features only, as the simplest baseline; Random Forest and Gradient Boosting consume the full window-based matrix; Ridge+Polynomial restricts itself to the top-15 window features by absolute correlation with the target, since a degree-2 expansion over ~ 260 columns is combinatorially explosive.

5 Models and Their Mathematics

5.1 RUL regression

Four baselines are trained under identical splits and features and compared on validation MAE.

Ridge regression. With feature matrix $X \in \mathbb{R}^{n \times p}$ and target y (the piecewise-capped RUL, Section 13.1),

$$\hat{\beta}_{\text{ridge}} = \arg \min_{\beta} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2, \quad \lambda = 1.0. \quad (7)$$

Ridge + Polynomial. A degree-2 expansion $\phi(x)$ of the top-15 correlated columns, fit with Ridge at $\lambda = 10.0$: $\hat{\beta}_{\text{poly}} = \arg \min_{\beta} \|y - \phi(X_{15})\beta\|_2^2 + 10\|\beta\|_2^2$.

Random Forest. $B = 60$ bootstrap-trained regression trees, depth capped at 6: $\hat{y}_{\text{RF}}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x)$.

Gradient Boosting (selected model). The lowest-validation-MAE model on *both* datasets, and therefore the deployed RUL model. It builds an additive ensemble of $M = 60$ shallow trees (`max_depth = 3`) by greedy stagewise fitting to the negative gradient of the squared-error loss $L(y, F) = \frac{1}{2}(y - F)^2$:

$$F_0(x) = \arg \min_{\gamma} \sum_i L(y_i, \gamma) \quad (8)$$

$$r_{im} = y_i - F_{m-1}(x_i) \quad (\text{negative gradient} = \text{residual under squared error}) \quad (9)$$

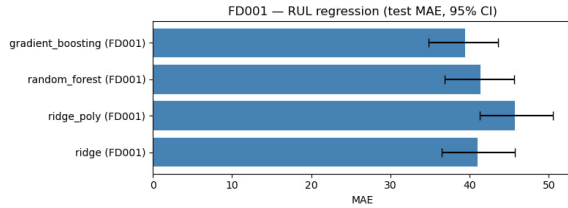
$$h_m = \arg \min_h \sum_i (r_{im} - h(x_i))^2 \quad (\text{tree fit to residuals}) \quad (10)$$

$$F_m(x) = F_{m-1}(x) + \nu \gamma_m h_m(x), \quad m = 1, \dots, M \quad (11)$$

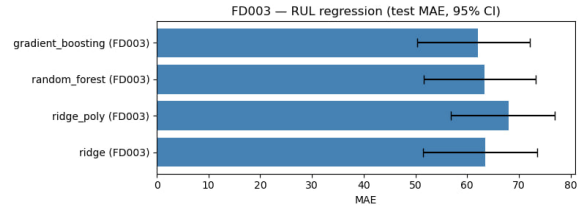
with learning rate $\nu = 0.1$ (scikit-learn default). Because r_{im} is exactly the current under/over-estimate of RUL, each new tree directly targets the model’s present error, which is why Gradient Boosting adapts well to the sharply nonlinear, saturating shape of a piecewise-capped RUL target.

Table 6: RUL regression – test set (95% engine-level bootstrap CI in brackets).

Model	FD001 MAE	FD003 MAE	Selected
Ridge	41.03 [36.4, 45.7]	63.55 [51.5, 73.5]	
Ridge + Poly	45.68 [41.3, 50.5]	68.00 [56.9, 77.0]	
Random Forest	41.34 [36.8, 45.7]	63.31 [51.6, 73.2]	
Gradient Boosting	39.45 [34.8, 43.6]	62.14 [50.4, 72.1]	✓



(a) FD001



(b) FD003

Figure 3: RUL regression test MAE with 95% engine-level bootstrap confidence intervals. Gradient Boosting is the narrowest and lowest bar on both datasets.

Breaking FD001 results down by life region confirms the model is most accurate exactly where it matters most for a maintenance decision:

Table 7: FD001 validation MAE by life region.

Region	n	MAE	RMSE
Early life	1,272	44.93	52.29
Mid life	1,280	14.85	18.51
Near failure	1,300	9.17	14.23

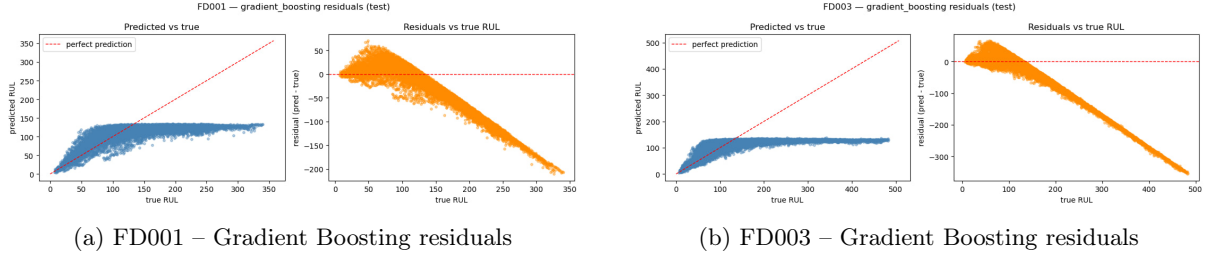


Figure 4: Predicted vs. true RUL on the test set. Both panels show predictions flattening near the cap value (130 cycles) for high true-RUL rows – by construction, a model trained on a capped target cannot predict much above the cap.

Feature importances from the winning Gradient Boosting model are dominated by rolling-mean and EWMA statistics of a handful of physically meaningful (temperature/pressure-adjacent) sensors, confirming that the window-based feature family adds real value over current-cycle values alone.

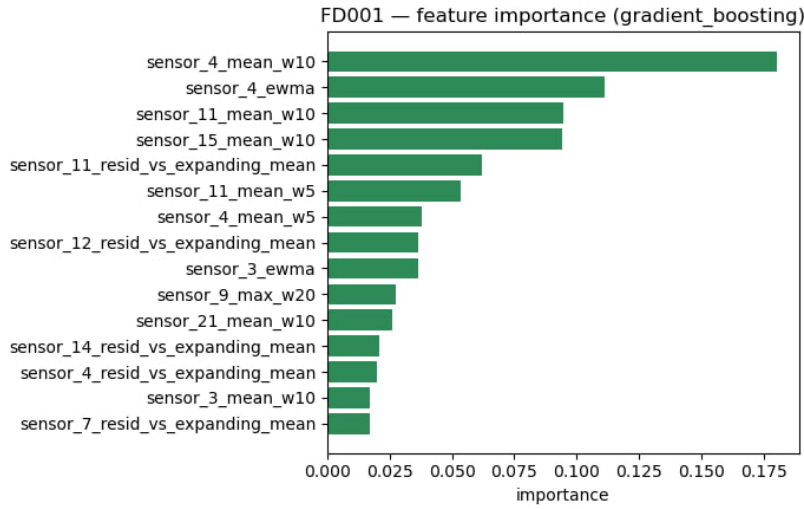


Figure 5: Top-15 Gradient Boosting feature importances, FD001.

5.2 Failure-horizon classification

Logistic Regression (selected model). For horizon h , with class-balanced weights ω_y to counter imbalance,

$$P(y_h = 1 \mid x) = \sigma(w^\top x + b) = \frac{1}{1 + e^{-(w^\top x + b)}}, \quad (12)$$

fit by minimizing the weighted log loss $\mathcal{L}(w, b) = -\sum_i \omega_{y_i} [y_i \log \sigma(w^\top x_i + b) + (1 - y_i) \log(1 - \sigma(w^\top x_i + b))]$. Compared against a same-architecture Gradient Boosting classifier, Logistic Regression wins on mean validation PR-AUC on both datasets (FD001: 0.973 vs. 0.966; FD003: 0.986 vs. 0.976) and is deployed, despite Gradient Boosting occasionally edging it out at a single horizon.

Cost-based threshold selection. Each horizon’s decision threshold is swept over $\{0.05, \dots, 0.95\}$ on validation predictions and chosen to minimize the asymmetric cost

$$C(\tau) = c_{\text{early}} \cdot \text{FP}(\tau) + c_{\text{miss}} \cdot \text{FN}(\tau), \quad c_{\text{miss}} = 10, \quad c_{\text{early}} = 1, \quad (13)$$

the same cost policy used in the end-to-end early-warning evaluation (Section 8).

Platt calibration. Raw probabilities are recalibrated with a sigmoid map fit on validation predictions only, $P_{\text{cal}}(y_h = 1 \mid x) = \sigma(A \cdot f(x) + B)$, where $f(x)$ is the base model’s score and A, B come from `CalibratedClassifierCV(method="sigmoid")`. Calibration reduces test-set Brier score at every horizon except one:

Table 8: Brier score before/after Platt calibration (test set).

Horizon	FD001			FD003		
	before	after	Δ	before	after	Δ
10	0.0012	0.0007	−42%	0.0002	0.0003	+50%
20	0.0039	0.0032	−18%	0.0022	0.0017	−23%
30	0.0089	0.0074	−17%	0.0064	0.0058	−9%

The one exception ($h=10$ on FD003) is an absolute change of 0.0001 on an already-tiny Brier score, driven by very few positive validation rows at that horizon.

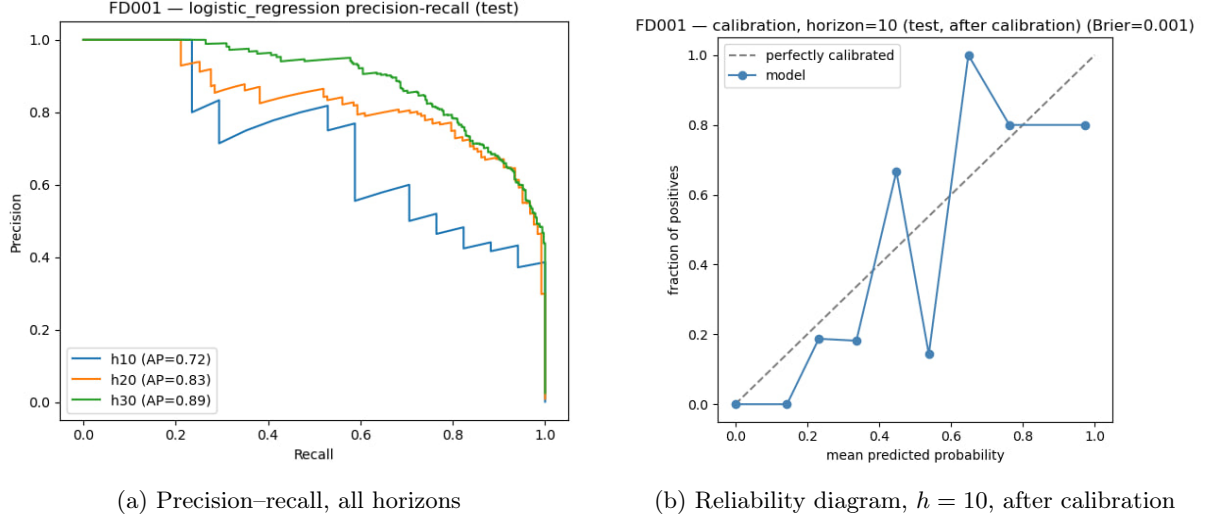


Figure 6: FD001 classification diagnostics for the selected Logistic Regression model.

5.3 Unsupervised anomaly detection

All four detectors are fit on a *healthy region* (first 30% of each training engine’s observed life) and scored so larger always means more abnormal.

Isolation Forest. $s(x, n) = 2^{-E[h(x)]/c(n)}$, $c(n) = 2H(n-1) - \frac{2(n-1)}{n}$, where $h(x)$ is path length and $H(\cdot)$ the harmonic number – shorter average path length gives a score closer to 1. 200 trees, contamination 0.05.

Local Outlier Factor (selected for FD001). For point x with k -neighborhood $N_k(x)$ ($k = 20$), local reachability density $\text{lrd}_k(x) = \left(\frac{1}{|N_k(x)|} \sum_{o \in N_k(x)} \text{reach-dist}_k(x, o) \right)^{-1}$, and $\text{LOF}_k(x) = \frac{1}{|N_k(x)|} \sum_{o \in N_k(x)} \text{lrd}_k(o) / \text{lrd}_k(x)$. Fit in `novelty=True` mode so held-out rows can be scored – required for leakage-safe test-time evaluation.

One-Class SVM. $f(x) = \sum_i \alpha_i K(x_i, x) - \rho$ from $\min_{w, \rho, \xi} \frac{1}{2} \|w\|^2 + \frac{1}{\nu n} \sum_i \xi_i - \rho$ s.t. $(w \cdot \Phi(x_i)) \geq \rho - \xi_i$, $\xi_i \geq 0$, $\nu = 0.05$; subsampled to 2,000 healthy rows since OCSVM scales poorly with row count.

PCA reconstruction error. $a_{\text{PCA}}(x) = \|x - WW^T x\|_2^2$, with $W \in \mathbb{R}^{p \times q}$ the top- q eigenvectors of the healthy-row covariance. q is chosen on validation data from $\{5, 10, 15, 20, 30, 50\}$ by most-negative correlation with validation RUL; $q = 50$ wins for both datasets.

Selection. Model choice uses Spearman correlation between each detector’s score and true validation RUL (want strongly negative):

Table 9: Spearman(RUL, anomaly score) per detector, validation set.

Detector	FD001	FD003
Isolation Forest	−0.710	− 0.746
LOF	− 0.716	−0.638
One-Class SVM	−0.696	−0.720
PCA reconstruction	−0.711	−0.682
Chosen	LOF	Isolation Forest

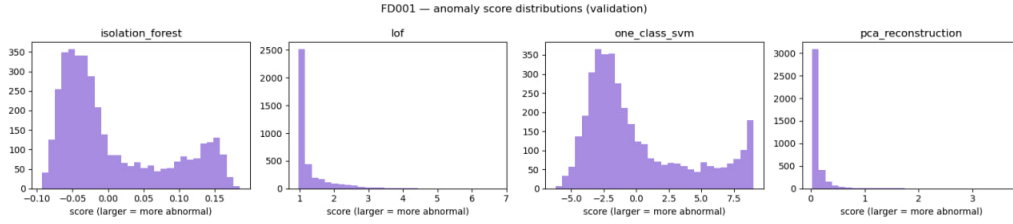


Figure 7: FD001 validation score distributions, all four detectors (larger = more abnormal).

6 Uncertainty and Calibration

6.1 RUL prediction interval

A Gaussian interval is built from the validation residual standard deviation $\hat{\sigma}$: $[\hat{y}(x) - 1.96\hat{\sigma}, \hat{y}(x) + 1.96\hat{\sigma}]$, with $\hat{\sigma} = 14.51$ (FD001) and computed analogously for FD003.

Table 10: RUL interval coverage on test (nominal 95%).

Dataset	Empirical coverage	Average width
FD001	51.2%	56.7 cycles
FD003	38.5%	52.0 cycles

Both datasets undercover their nominal target substantially. This is expected: a single Gaussian band built from in-sample validation residuals cannot represent the strongly heteroscedastic, right-skewed error visible in Figure 4 (near-failure rows are tight; early-life, above-cap rows carry larger, biased error). A conformal or per-life-region interval is the natural next iteration; we flag this as a known limitation rather than reporting only the nominal target.

6.2 Classifier calibration and anomaly normalization

The $h=10$ decision threshold is re-tuned on the *calibrated* probability, so the deployed threshold and probability live in the same space. Raw anomaly scores are never surfaced directly: each detector stores its validation score distribution, and at inference time a raw score is converted to a validation percentile, $\text{pct}(a) = \frac{|\{a' \in \text{val. scores} : a' \leq a\}|}{|\text{val. scores}|} \times 100$, which the dashboard and decision policy consume.

7 Decision Policy

The three signals are fused by a single `recommend()` rule, shared byte-for-byte between the training notebook and the deployed app:

STOP if $\text{RUL}_{\text{lower}} \leq \tau_{\text{RUL}}^{\text{stop}}$ or $P(\text{fail} \leq 10) \geq \tau_{\text{risk}}^{\text{stop}}$
 INSPECT else if $\text{RUL}_{\text{lower}} \leq \tau_{\text{RUL}}^{\text{insp}}$ or $P(\text{fail} \leq 10) \geq \tau_{\text{risk}}^{\text{insp}}$ or $\text{pct}(a) \geq \tau_{\text{anom}}^{\text{insp}}$
 CONTINUE otherwise.

$\tau_{\text{RUL}}^{\text{stop/insp}}$ are the 10th/30th percentiles of validation RUL; $\tau_{\text{risk}}^{\text{stop}} = 0.8$; $\tau_{\text{risk}}^{\text{insp}}$ is the calibrated $h=10$ threshold; $\tau_{\text{anom}}^{\text{insp}} = 90\text{th percentile}$.

Table 11: Deployed decision policy per dataset.

Parameter	FD001	FD003
$\tau_{\text{RUL}}^{\text{stop}}$	19.0 cycles	23.0 cycles
$\tau_{\text{RUL}}^{\text{insp}}$	57.0 cycles	69.0 cycles
$\tau_{\text{risk}}^{\text{stop}}$	0.80	0.80
$\tau_{\text{risk}}^{\text{insp}}$	0.10	0.10
$\tau_{\text{anom}}^{\text{insp}}$	90th pct.	90th pct.

Every trigger produces a human-readable reason string (e.g. “*RUL lower bound (36.7) $\leq$ inspect threshold (69.0)*”), and a disagreement between signals – elevated anomaly percentile with low supervised risk – is surfaced explicitly rather than silently resolved.

8 Early-Warning Analysis and Asymmetric Cost

For each held-out engine i , the first *persistent* alert cycle $\tau(i)$ (requiring m alerts within the last n cycles, not one noisy flag) defines lead time $L(i) = T(i) - \tau(i)$, and the operational cost is

$$C = c_{\text{miss}} \cdot \mathbb{I}[\text{missed}] + c_{\text{late}} \cdot \max(0, h - L(i)) + c_{\text{early}} \cdot \max(0, L(i) - h), \quad (14)$$

with $c_{\text{miss}} = 10 > c_{\text{late}} = 3 > c_{\text{early}} = 1$ and $h = 10$.

Table 12: Early-warning cost, tuned vs. naive threshold, validation engines.

Dataset	Miss rate	Miss/Late/Early cost	Tuned total	Naive (0.5) total
FD001	0%	0 / 33 / 49	82	95
FD003	0%	0 / 24 / 34	58	84

Zero missed warnings on both datasets is the key result: the tuned threshold trades a small amount of early-warning burden for a guarantee that no failure goes unflagged, consistent with $c_{\text{miss}} \gg c_{\text{early}}$. Cost-based tuning beats the naive 0.5 cut-off by 14% (FD001) and 31% (FD003).

9 Monotonicity and Trend Diagnostics

As a sanity layer beyond aggregate metrics, every validation engine’s RUL trace is checked for how often predicted RUL rises with cycle (it should almost always fall), and its Spearman correlation with cycle is reported. On FD003, all 20 validation engines are flagged (mean violation rate 0.463, mean Spearman -0.937); mean Spearman between RUL and $P(\text{fail} \leq 10)$ is -0.925 , and between RUL and the anomaly score is -0.859 .

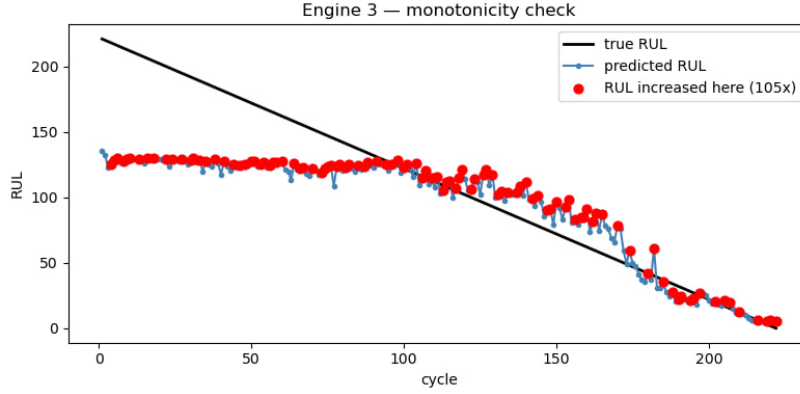


Figure 8: FD003 engine 3 – predicted RUL trace with monotonicity violations marked. The plateau near the top of the curve is the RUL cap in action: once true RUL exceeds ~ 130 cycles, the model has no incentive to distinguish 200 from 300 cycles remaining, so predictions compress into a narrow, mildly noisy band – where most “violations” in this diagnostic come from.

10 Stage 2 Findings: FD001 vs. FD003

Reusing one locked notebook structure, split policy, feature pipeline, and evaluation protocol for both datasets makes Table 13 a fair comparison.

Table 13: Master comparison, FD001 vs. FD003, identical protocol.

	FD001 (1 fault mode)	FD003 (2 fault modes)
RUL MAE (Gradient Boosting)	39.45	62.14
RUL R^2	0.69	0.37
Classification PR-AUC, mean	0.97 (LogReg)	0.99 (LogReg)
Anomaly Spearman(RUL, score)	−0.716 (LOF)	−0.746 (Iso. Forest)
Early-warning tuned cost	82	58
Interval empirical coverage	51.2%	38.5%

RUL regression degrades markedly on FD003 (R^2 drops from 0.69 to 0.37) even though classification, if anything, improves – distinguishing “failing soon” from “not yet” is coarser and easier than pinning down exactly how many cycles remain when two fault modes with different progression rates share one training set with no fault-mode label. The winning anomaly detector also changes (LOF for FD001, Isolation Forest for FD003): FD003’s two fault modes appear to form more separated, tree-friendly clusters, while FD001’s single degradation pattern favors a density-based method.

11 Dashboard and Deployment

The system is deployed as a Streamlit app (`app/app.py`) reading exactly the artifacts the notebook exports (`feature_pipeline.pkl`, `rul_suite.pkl`, `clf_suite.pkl`, `anomaly_suite.pkl`, `decision_policy.json`, `metadata.json`), guaranteeing *app parity*: the dashboard’s inference path is the same `PrognosticsSystem` object exercised in the notebook, not a reimplement. Selecting a dataset and engine renders an engine timeline, an RUL card (point estimate, 95% interval), a failure-risk card (calibrated 10/20/30-cycle probabilities), an anomaly card (percentile and raw score), and an action banner (CONTINUE/INSPECT/STOP) with the exact trigger reason.

Model bundles and the decision logic live in importable `src/` modules rather than being defined inline in the notebook, so that `joblib` records their class location as a real module (`src.model_bundles`, `src.decision`) instead of the notebook’s transient `__main__` namespace – otherwise the deployed app fails to unpickle the models with an `AttributeError`. This module boundary is what makes the notebook and the deployed app run byte-identical inference logic.

12 Error Analysis

- **Early-life RUL error dominates.** MAE falls from 44.9 (early life) to 9.2 cycles (near failure) on FD001 – exactly the region where a maintenance decision matters least, since no action is warranted hundreds of cycles before failure regardless of the exact number.
- **FD003’s mixed fault modes inflate RUL error without hurting classification.** See Section 13 for the cap-related discussion.
- **Interval undercoverage.** Both datasets’ 95% nominal Gaussian RUL band achieves well under 95% empirical coverage (51.2% / 38.5%), concentrated in the early-life region where residual variance is highest; flagged as an open item rather than tuned away post hoc, since re-tuning on test data would itself be a leakage violation.
- **Anomaly detection is slightly harder on FD003.** FD003’s lower Spearman correlation for the LOF/OCSVM/PCA detectors (Table 13 region) indicates the second fault mode adds noise that only the tree-based Isolation Forest fully compensates for.

13 Challenges

13.1 Selecting the RUL cap

A piecewise-capped RUL target is standard practice in C-MAPSS to avoid distinguishing engines comfortably far from failure. Initially, grid-searching the optimal cap yielded structural bias: smaller cap values artificially shrank the target range, mechanically lowering MAE regardless of generalization performance. Consequently, we adopted a fixed cap of 130 cycles (near FD001’s 90th percentile RUL) applied uniformly to ensure a fair comparison.

In FD003’s complex multi-fault regime, RUL predictions flatten for true RULs exceeding this cap (Figure 4), reducing lower-bound discrimination power. To avoid relying on unreliable high-RUL resolution, our decision policy adopts a conservative approach: when predicted lower bounds enter the capped threshold, the system triggers a STOP recommendation rather than attempting fine-grained subdivision. Although this slightly increases inspection overhead, FD003 maintains a lower overall early-warning cost than FD001 (58 vs. 82 at 0% miss rate), prioritizing auditable safety over unsupported precision.

13.2 Deferring FD004 Due to High Computational Cost

The assignment brief lists FD004 (six operating conditions, two fault modes) as an optional bonus, requiring only FD002 *or* FD003 for Stage 2. We initially attempted to include FD004 to tackle the most complex subset directly.

However, running the complete pipeline on FD004 required over **60 minutes of computation time** per run—driven by nearly $2.5\times$ more training engines (248 vs. 100) and multi-condition complexity. Given the fixed timeline, spending an hour per run was impractical for iterative validation and threshold tuning. We therefore locked in FD003 as our Stage-2 benchmark, cleanly isolating multi-fault-mode challenges while preserving an efficient, fully audited comparison.

14 Conclusion

The final system meets every mandatory requirement of the brief: a leakage-safe, engine-level split enforced from the first cell; causal, tested feature engineering; four compared RUL baselines with Gradient Boosting selected on validation MAE and evaluated by life region; three calibrated failure-horizon classifiers with cost-based threshold tuning; four anomaly detectors compared on a common, validation-normalized scale; an auditable decision policy that exposes its own uncertainty and disagreement; and one identical protocol reused across the FD001 foundation and FD003 challenge stages. The two documented challenges – replacing a biased automatic cap search with a single defensible manual cap, and recognizing FD004 as a scope risk before it consumed the project timeline – were the two decisions that most shaped whether this system ships as genuinely auditable rather than merely accurate on paper.